

Cloud Vulnerability Scanner

A comprehensive, containerized security scanning tool designed to identify vulnerabilities in cloud infrastructure (AWS) and web applications. It integrates multiple open-source security tools into a unified dashboard.

Features

- **Multi-Cloud Support:** Scans AWS environments (IAM, EC2, S3) and GCP Projects.
- **Kubernetes Support:** Scans container images and performs local Kubernetes cluster vulnerability assessments.
- **Web App Scanning:** Integrated OWASP ZAP and Nuclei for web vulnerability detection.
- **Secret Scanning:** Detects hardcoded secrets using Gitleaks.
- **Dependency Analysis:** Checks for vulnerable dependencies with Trivy and Safety.
- **Unified Dashboard:** View all findings in a single, easy-to-use web interface.
- **Dockerized:** Fully containerized for easy deployment.

Prerequisites

Before you begin, ensure you have the following installed: - [Docker Desktop](#) (includes Docker Compose) - [Git](#)

Installation & Setup

1. Clone the Repository

```
git clone <your-repo-url>
cd Cloud_Vulnerabilities
```

2. Configure Environment Variables

Create a `.env` file in the root directory. You can start by copying the example (if available) or ensuring you have the following keys:

```
# Database (PostgreSQL)
POSTGRES_USER=scanner_user
POSTGRES_PASSWORD=scanner_pass
POSTGRES_DB=scanner_db

# Redis
REDIS_PASSWORD=redis_password

# Application Secrets
SECRET_KEY=your-super-secret-key-change-this
```

```

ENCRYPTION_KEY=your-encryption-key

# AWS Configuration (Required for Cloud Scans)
# You can also map your ~/.aws folder in docker-compose.yml
AWS_ACCESS_KEY_ID=your_access_key
AWS_SECRET_ACCESS_KEY=your_secret_key
AWS_PROFILE=company
AWS_SDK_LOAD_CONFIG=1
AWS_REGION=us-east-1

# GCP Configuration (Optional)
GCP_PROJECT_ID=your_project_id
GCP_SERVICE_ACCOUNT_JSON=/app/config/gcp-service-account.json

# AI Analysis (Optional)
OPENAI_API_KEY=your_openai_key

# Public Deployment (HTTPS)
DOMAIN=your-domain.com
CADDY_EMAIL=you@example.com

```

3. Setup Cloud Permissions (AWS)

To scan your AWS account, the scanner needs specific **Read-Only** permissions. 1. Navigate to the `infra/` folder. 2. Use the provided CloudFormation template or `iam_permissions.json` to configure an IAM Role or User with the necessary permissions. - See `infra/README.md` for detailed instructions on deploying the permissions stack. 3. Any resource created through the provided AWS templates/examples should carry the tag `Owner=cloudvul@iitm`. 4. To switch the AWS account used by local Docker-based scans, set `AWS_PROFILE` to the company profile before starting the stack.

Usage

Start the Application

Run the entire stack using Docker Compose:

```
docker compose up --build
```

This will start: - **Backend API**: `http://localhost:8000` - **Frontend Dashboard**: `http://localhost:80` - **Database & Redis**: Background services

Access the Dashboard

Open your browser and navigate to: <http://localhost>

Running a Scan

1. Log in to the dashboard.
2. Configure your target (URL, IP, or Cloud Account).
3. Click “**New Scan**”.
4. View real-time progress and detailed reports.

Optional Tooling For Stronger IaC And Container Coverage

The new **IaC Security** and **Container Security** flows automatically use any of these CLIs when present on the host:

- IaC: trivy, checkov, tfsec, semgrep, gitleaks
- Container: trivy, gype, snyk, syft, hadolint

Examples:

```
# IaC scanners
```

```
pip install checkov semgrep
choco install tfsec
```

```
# Container scanners
```

```
choco install trivy gype syft hadolint
npm install -g snyk
```

Notes: - **snyk** requires prior authentication with **snyk auth** - **syft** is used as an SBOM fallback when Trivy SBOM generation is unavailable - the dashboard tool summary reflects whichever scanners actually produced findings

Architecture

- **Frontend:** HTML/JS Dashboard (served via Nginx)
- **Backend:** FastAPI (Python)
- **Database:** PostgreSQL (Findings storage)
- **Queue:** Redis (Task management)
- **Tools:** Trivy, Gitleaks, Nuclei, CloudFox, OWASP ZAP

SSO Integration

The active DeepTrustxAI SSO frontend now lives in:

- [sso/new/deeptrustxai](#)

That app provides:

- Google sign-in with NextAuth
- signed SSO token handoff for CloudGuard
- the **Cloud Penetration Testing** product tile that opens CloudGuard after login

CloudGuard expects these companion SSO routes:

- `/api/scan/redirect`
- `/api/sso/verify`

The older folder:

- `sso/sso-login`

is now a legacy implementation and should not be used as the primary SSO app for new deployments.

Security Note

Never commit your `.env` file. It contains sensitive API keys and secrets. This repository includes a `.gitignore` to prevent accidental commits, but always double-check.

Deploy Online (Production + HTTPS)

This project includes a production profile with Caddy reverse proxy (automatic HTTPS).

1. DNS + Firewall

- Point your domain A record to your server public IP.
- Open inbound ports: 80 and 443.

2. Set deployment env vars

In your `.env` (or `.env.prod`) set: - `DOMAIN=your-domain.com` - `CADDY_EMAIL=you@example.com`
- `SECRET_KEY`, `ENCRYPTION_KEY`, cloud credentials, DB/Redis passwords

3. Start production stack

```
docker compose --profile prod up -d --build postgres redis backend caddy
```

4. Verify

```
docker compose --profile prod ps
curl -I https://your-domain.com
```

5. Security defaults included

- PostgreSQL port bound to localhost only.
- Redis port bound to localhost only.
- Backend is no longer exposed directly on host port in production.